IT3708: Project 1

Feature Selection with Genetic Algorithms

Lab Goals

- Implement a genetic algorithm to select the most impactful features in a dataset.
- Implement crowding in the genetic algorithm to increase diversity within the population.
- Show the results of the feature selection, and compare results when using crowding, no crowding, and not doing any feature selection.
- Use a generalization of the knapsack problem as a stepping stone towards the feature selection problem.
- Prepare yourself for using high-performance languages such as Julia, Java, C++ or Rust which you will need in Project 2 and Project 3.
- Learn to take advantage of libraries for evolutionary computation, such as EvoLP for Julia which was developed here at NTNU.

Important: This project is graded (15 points out of 70 points for the projects) and we highly recommend doing it. The first part of the project you have to complete and deliver by yourself, to help everyone get started programming bio-inspired artificial intelligence in a high-performance language. In the second part of the project, you have to form groups and work together. In the rest of the semester, you will be completing large, difficult programming projects. It is highly beneficial to find a reliable group and get used to working together early.

Group size. We allow up to 3 students in each group. You can keep the same group for each project, or switch groups if you discover the group does not work for you. All students who participated must be present during the demo day. All members of a group should sign up for the same time slot on demo day. If you absolutely insist for any reason, you may work alone, but we will treat you as a full-sized group. This is true to work life where not being able to work with others is disadvantageous.

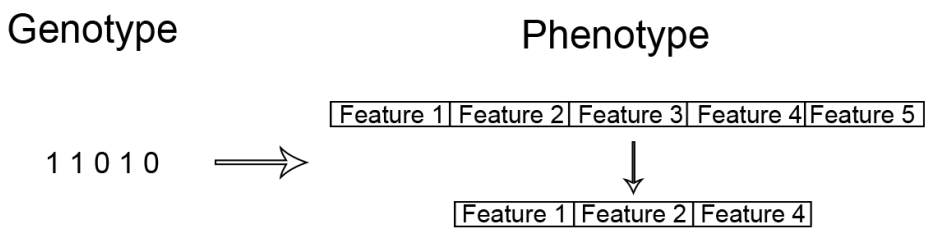
Programming language: Though this first project can be solved with standard Python, we recommend using a high-performance language. We know people have had previous success with **Julia** (our recommendation), **Java**, **C++**, **Rust** and Go (At least one of the staff has experience with one of the marked programming languages). The choice of programming language will be essential for the next two

projects, and you probably have more time early in the semester if you have not used these languages in a while (or never before). You may be able to make Python faster using Cython and other typing or compilation tricks, but this is not a trivial approach. We will give some introduction to Julia early in the semester during lab hours. Julia is designed to be similar to Python while remaining competitive in performance.

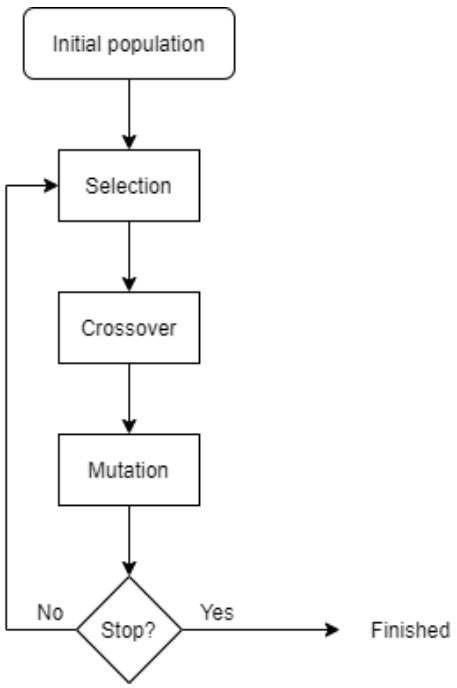
Deadline: February 9th, 2026

Introduction to Genetic Algorithms

To solve the problems in the first project, you will implement a genetic algorithm (GA) as presented in lectures and the literature. The so-called Simple Genetic Algorithm (SGA) is an excellent starting point [1] [2] [3]. It is recommended to represent the individuals as bitstrings. In the feature selection problem, each bit decides whether a feature should be included or not.



Note that GA parameter values (population size, number of generations, crossover rate, mutation rate, etc.) are important to optimize and are typically correlated. Your GA may successfully find optimal or near-optimal solutions if you use reasonable parameter values. However, there is no definite rule as to how to find such parameter values. Therefore, you should test different sets of parameter values to decide on appropriate values. You are advised to closely study the interdependence between the parameter values in your chosen implementation.



There are many variants of genetic algorithms. In this project we will focus on two types: the simple genetic algorithm (SGA) [1] [2] [3] and the crowding algorithm [2] [4]. The two approaches are quite similar, but with some small but important differences in the replacement step.

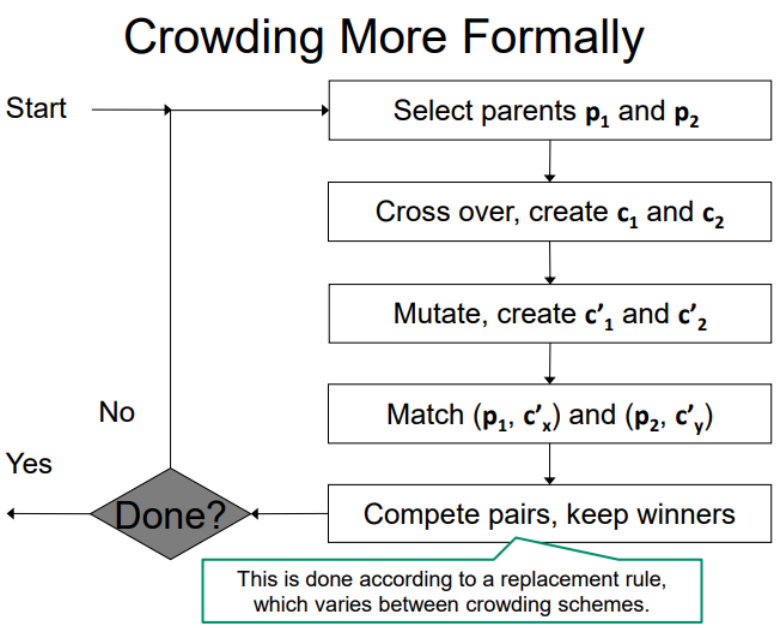
GA 1: The Simple Genetic Algorithm

The simple genetic algorithm [1] [2] [3] consists of generating an initial population, selecting the best parents of the offspring, doing crossover and mutation, and replacing the population. This process is repeated until an end condition is reached.

GA 2: The Crowding Concept

A problem that may arise when using an evolutionary algorithm is having most or all individuals in the population converging to the same solution. This might be undesirable, because there might be better solutions that the algorithm is not finding due to premature convergence.

Crowding is a technique that can be applied to extend the SGA and achieve better performance. It combats the convergence to similar solutions by increasing diversity within the population [2] [4]. The survival step in the genetic algorithm is replaced with a local tournament, for example between a parent and its closest child. There are different variants of crowding, including De Jong's scheme, deterministic crowding, probabilistic crowding, and restricted tournament selection [2] [4]. In this project you are asked to implement and demonstrate one or two of these crowding methods (see below).

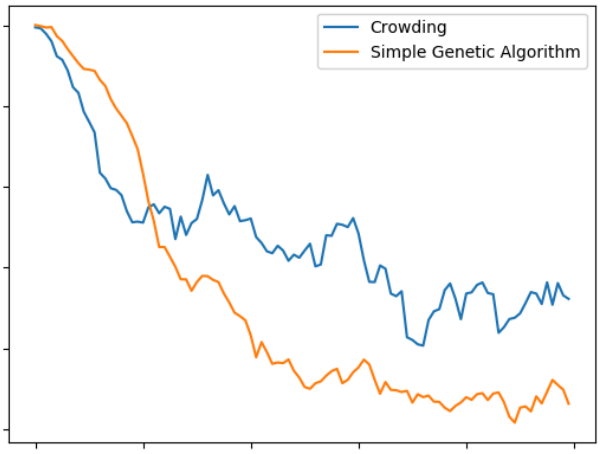


Population Diversity and Entropy

As explained above, it can be desirable to encourage a certain level of diversity or variation within the population of the genetic algorithm. To measure this variation, we can use the entropy of the population, given by the formula:

$$H = -\sum_i p_i log_2 p_i \ .$$

Here, *i* denotes the index in the bitstring, and p_i denotes the probability of the *i*th bit being a 1. The probability is estimated from the population.



The figure shows a comparison of the entropies (on the *y*-axis) over the generations (on the *x*-axis) for implementations of a crowding GA and a simple GA. The figure suggests that the Crowding GA maintains diversity, as measured by entropy, better than the simple GA with increasing generations.

Theory Questions (3 points)

During demo day the group may be asked some questions. Some examples:

a) What is niching in the context of evolutionary algorithms, and is it helpful in this project?
b) What is the difference between the genotype and phenotype in this project?

Note, you will probably not be asked exactly these questions, and the questions may be more tailored to your particular implementation and demonstration. Further, the number and/or difficulty of the questions may be greater for larger teams.

Part 1: The binary knapsack problem (to be completed individually)

Important: This part of the project you have to complete and deliver individually. We will check that your code is different from the work of other students.

In this project we will be implementing two different fitness functions, both of which operate on bit strings. Instead of directly attacking the feature selection problem, we

will first study a synthetic or artificial problem. The practice of using simpler synthetic functions as stepping stones for more complex real-world problems is common in artificial intelligence theory and practice [2] [3] [4]. The GA stays essentially the same, perhaps some of the GA parameters need to be set differently, but the fitness function is faster to evaluate and easier to understand than natural fitness functions.

Fitness Function 1: The binary knapsack problem

The first fitness function we will be using is a well-known problem in optimization: the binary (or 0/1) knapsack problem: given a container with a limited capacity and a set of items with different values and weights, what is the subset of items that fit in the container that maximizes the obtained value? Formally, this problem looks like the following.

Problem definition. Given a knapsack of capacity *c* and a set of *n* items with profits p_j and weight w_j of item *j*; select a subset of the items in order to:

maximize $z = \sum_{j=1}^{n} p_i x_j$

subject to $\sum_{j=1}^{n} w_j x_j \leq c,$

$x = 0 \; or \; 1, \qquad j \in N = \{1, \cdots, n\}$

where

$x_j = 1$ if item *j* is selected, or $x_j = 0$ otherwise.

Here, the task will be to maximize the sum of the values *z* of the items (*p*) inside the container. A chromosome *x* would then be a bitstring where 1 or 0 means if the item should be packed or not. A *dataset* is provided as part of the project assignment, and is presented below.

Dataset. The dataset to be used with the knapsack problem is based on one of David Pisinger's hard knapsack instances [7], specifically the file `knapPI_12_500_1000_82.csv`. It features a problem with **500 different items** (one item per row), and a **knapsack capacity of 280785 units**. The **optimal solution is 296835**, and although it is very unlikely to find it using a GA, you can achieve solutions which are quite close.

Indirect Constraint Handling with Penalty Functions

In the knapsack problem, the container has a maximum capacity that cannot be exceeded, so the sum of all packed item-weights needs to be less or equal than this capacity *c*. Such constraints can be handled indirectly by adding a penalty term to the fitness function. The original fitness function $f(x)$ is then transformed into $f'(x) = f(x) + P(d(x,F))$, where $d(x,F)$ is a distance metric of the infeasible point to the feasible region F. The penalty function P is zero for feasible solutions and increases proportionally with the distance to the feasible region [1].

Part 1, Programming (5 points)

For the programming tasks, we recommend using a high-performance language such as Julia, Java, C++ or Rust. See the note at the beginning of the document for more information. We welcome you to use evolutionary computation libraries in your code. For example, EvoLP for Julia which was developed here at NTNU. When implementing the different functions it may be a good idea to keep the GA parameters in one place so you can easily run variations of the algorithm.

a) Use the provided dataset with the item weights and profits to implement the knapsack fitness function in your code. This should handle the capacity constraint and penalize unfeasible solutions. If you are using a library, like EvoLP, note that operators might usually minimize by default. **(1 point)**

b) Program your own population generators, parent selectors, crossover methods and mutators. You may also use a library, such as EvoLP. In that case, you can familiarize yourself with its available operators. Either way, combine all your operators and test your fitness function. **(1 point)**

c) Implement two survivor selection functions that select the survivors of a population based on their fitness. **(1 point)**

d) Connect all the implemented functions and suitable operators to complete your own genetic algorithm with survival selection, and run the algorithm with the knapsack fitness function. Throughout the generations plot the maximum, minimum and mean fitness values of the population. **(1 point)**

e) This task is identical to *d)*, however, we now aim to find a feasible solution x^* that is close to the known optimum (**$x^* > 29000$**). **(1 point)**

We ask you to **individually deliver unique code** which, when being run, neatly prints the results of the above tasks and shows that you have completed them.

Part 2: Feature Selection (group work)

Important: This part of the project you have to complete and present in group. You may reuse code from Part 1 written by any member of your group.

Feature selection is a machine learning process where redundant and irrelevant features are removed from a dataset [5]. Evolutionary algorithms including genetic algorithms have been successfully used for the purpose of feature selection [6]. The selected features will often provide approximately as good, or better, results, and having data with fewer dimensions has some advantages when training a predictive model:

- Faster training time
- Reducing overfitting
- Requiring less data entries

In this project you are tasked with creating a genetic algorithm for finding the best features for a given dataset. The features will be passed to a simple machine learning algorithm which returns the root-mean-square error. This root-mean-square error can in turn be used as a fitness score for the implementation of the genetic algorithm. The objective is to minimize the root-mean-square error.

Fitness Function 2: Feature Selection

The second task will be related to the feature selection task described above, and is in fact an application of the binary knapsack problem (with a relaxation on the capacity constraint). Two key components of this task are a *dataset* as well as a *machine learning algorithm*. Both of these components are provided as part of the project assignment and are also presented below.

Machine learning algorithm. We provide the algorithm in the files LinReg.jl (for Julia) and LinReg.py (for Python). If you are working in another language, you should be able to reproduce the code in your own language with not much difficulty. The Python file may be easier for you to translate than the Julia code. Major programming languages are likely to have similar libraries available.

The files contain a function that will run a simple machine learning algorithm (linear regression) for a dataset, and return the root-mean-squared error. This error can be used as a fitness function for your genetic algorithm. It is important to keep in mind that your genetic algorithm should attempt to minimize the root mean squared error, and not maximize it. You do not need to understand the details of linear regression in order to use the root mean squared error as a fitness function.

In order to use the **LinReg.jl** file, you will need MLJ and MLJLinearModels. We also recommend looking at the CSV, DataFrames and StableRNGs packages, as they will make your work easier. In order to use the **LinReg.py** file, you will need Numpy and scikit-learn. The functions in LinReg.py are the same, but take different parameters than in LinReg.jl. The following functions are provided in LinReg.jl:

- ***get_fitness(model, X,y)***: Gives the root-mean-square error achieved by ***model*** on the data ***X*** with targets ***y***.
- ***get_columns(X, bitstring)***: Selects features from data ***X*** based on the bits in ***bitstring***.

You can learn more about the methods by looking at the documentation in the code.

Dataset. The dataset to be used for machine learning and feature selection consists of 1994 rows each having 102 columns. The first 101 columns represent the data, while the last column represents the value of the row. You will have to separate the targets **y** from the features **X** when you load the data.

Part 2, Programming (7 points)

f) Run your genetic algorithm on the provided dataset. Show the results, and compare them to the results of not using any feature selection (given by running a linear regression with all features selected). **(1 point)**

g) By playing with the parameters, how much are you able to improve the RMSE? What is the lowest and highest RMSE you are able to get with different parameters? Why do you think you get particularly low or particularly high RMSE with certain parameters? Note that the RMSE range may be different on different computers due to different random seeds in LinReg. **(1 point)**

h) Implement two new survivor selection functions. The functions should be using a crowding technique as described in the section about crowding. Do exercise d) from Part 1 and f) again with the new selection functions, and compare the results to using the simple genetic algorithm. Also show and compare how the entropies of the different approaches (SGA and crowding) change through the generations through a plot. **(1 point)**

i) Convergence can be accelerated by implementing survivor selection with elitism at the cost of reducing diversity. If you did not do so already in c), implement one new survivor selection function with elitism. Compare the convergence and diversity against a GA using crowding approaches. **(1 point)**

We ask you to, along with your group, **deliver a report** (no more than 3 pages) answering these tasks. We will also ask you to present your results on **demo day**. The quality of your report and presentation may earn you another **3 points**.

Delivery Method and Deadline

There are **3 deliverables**: a) Your individual code for Part 1, b) the group report and c) the group's shared code. You must also give a brief presentation during **demo day**. In total, you may earn up to 15 points for this project: **3 points** for theory questions during the presentation, **5 points** for code solving Part 1, **4 points** for shared code solving Part 2 and another **3 points** for the quality of your report and presentation.

- **Individual code**: You should upload your code directly to BlackBoard (not in a zip file). There will be a separate place on BlackBoard to deliver. You should also include a screenshot showing you got the results we asked for. **Note:** In case we want to run it ourselves, your code has to be able to run to completion within a reasonable time.
- **Report**: You should deliver a group report for Part 2 (no more than 3 pages), summarizing your experimental setup, the results of your experiments, your findings and your conclusions. Remember to include your references and **cite them properly**.
- **Group's shared code**: You should deliver this alongside the report in a zip file.

The submission system will be closed on **February 9th**. The **demo day** for Project 1 is also **February 9th**. A signup schedule will be announced one week before. Please follow Blackboard for details. **In addition to their individual work, every student must submit a copy of their jointly developed code and report. Every member of the group must attend on the scheduled demo date. No early or late submission or demo will be entertained except in case of an emergency.**

References

[1] A. E. Eiben and J. E. Smith. "Introduction to Evolutionary Computing," 2nd Edition, Springer 2015, pages 99 - 100, Table 6.1 (SGA) & pages 91 - 95 (population diversity, crowding) & pages 203 - 211 (constraint handling).
[2] D. Simon. "Evolutionary Optimization Algorithms," Wiley 2013, pages 44 - 55 (SGA) & pages 192 - 198 (population diversity, crowding).
[3] D. E. Goldberg "Genetic Algorithms in Search, Optimization, and Machine Learning," Addison-Wesley, 1989, pages 59-75.

[4] O. J. Mengshoel and D. E. Goldberg. 2008. “The Crowding Approach to Niching in Genetic Algorithms.” Evol. Comput. 16, 3 (Fall 2008), pages 315–354. DOI:https://doi.org/10.1162/evco.2008.16.3.315
[5] I. Guyon and A. Elisseeff. 2003. “An Introduction to Variable and Feature Selection.” Journal of Machine Learning Research 3, (3/1/2003), pages 1157–1182.
[6] B. Xue, M. Zhang, W. N. Browne and X. Yao, "A Survey on Evolutionary Computation Approaches to Feature Selection." IEEE Transactions on Evolutionary Computation, vol. 20, no. 4, pages 606-626, Aug. 2016, doi: 10.1109/TEVC.2015.2504420.
[7] D. Pisinger, ‘Where are the hard knapsack problems?’, *Computers & Operations Research*, vol. 32, no. 9, pp. 2271–2284, Sep. 2005, doi: 10.1016/j.cor.2004.03.002.
[8] S. S. Skiena. 2008. The Algorithm Design Manual. 2nd ed. Springer-Verlag.

Annex

Some other useful resources for Julia

- The Julia Programming Language documentation: https://docs.julialang.org/en/v1/
- An excellent book on optimization algorithms by Kochenderfer and Wheeler: Algorithms for Optimization
- EvoLP documentation: https://ntnu-ai-lab.github.io/EvoLP.jl/stable/
- MLJ documentation: https://juliaai.github.io/MLJ.jl/dev/